

13

/ 13 LECTURES

The Complete Internet Journey

One HTTPS request, end to end — browser cache, DNS, ARP, TCP, TLS, HTTP, NAT, routing, load balancers, and the ride home. Every lecture, one story.

Browser Cache

DNS Lookup

ARP

TCP Handshake

TLS Handshake

Diffie-Hellman

HTTP Request

NAT & Routing

Load Balancer

Response & Close

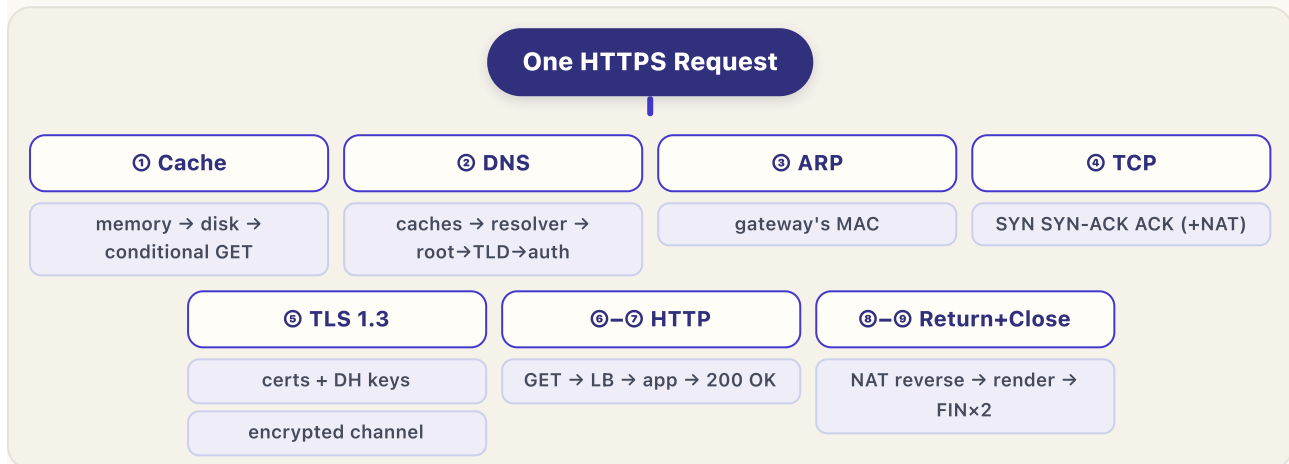
Full Timeline

WHAT THIS LECTURE GIVES YOU

The course payoff: follow one request from typing `https://www.example.com` to pixels on screen — all nine phases, the millisecond-by-millisecond timeline, TLS 1.3 and the paint-mixing Diffie-Hellman analogy, where every protocol from Lectures 1–12 appears, plus a troubleshooting map of everything that can go wrong.

LEARNING OBJECTIVES — AFTER THIS LECTURE YOU CAN

- Trace an HTTPS request **end-to-end** through all nine phases
- Map every phase to its **OSI layer and protocol**
- Explain the **TLS 1.3 handshake** and Diffie-Hellman key exchange
- Identify **bottleneck phases** and their optimisations (cache, CDN, keep-alive)
- Connect **all 12 previous lectures** into one mental model



The scenario — pinned numbers used throughout

URL: <https://www.example.com/products?id=42> · Laptop 192.168.1.5 (DHCP) · GW 192.168.1.1 · Router WAN 203.0.113.5 · Server 93.184.216.34 (behind a load balancer)

1 Phase 1 — Browser Cache (0 ms if lucky)

1 Memory cache

Same browsing session & fresh? Return instantly — zero network.

2 Disk cache

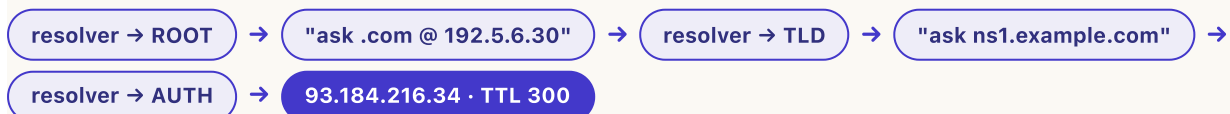
Within Cache-Control: max-age? Serve from disk. Stale? Send a **conditional GET**: If-None-Match: "etag123" → server replies **304 Not Modified** → reuse cached body (no transfer!).

3 Miss → full journey begins

All the headers that make this work: Cache-Control (freshness), ETag (fingerprint), Expires (legacy).

2 Phase 2 — DNS Resolution

Cache chain first: browser → OS → /etc/hosts → router forwarder (192.168.1.1:53) → recursive resolver (8.8.8.8) → hierarchy:



The DNS packet itself

UDP · src port 54321 (ephemeral) · dst port 53 · query type A — answer cached 300 s by every hop that touched it.

3 Phase 3 — ARP the Gateway

Routing decision: 93.184.216.34 $\notin$ 192.168.1.0/24 → use gateway 192.168.1.1
ARP cache: hit → router MAC 11:22:33:44:55:66 (miss → broadcast + reply)
Frame built: DstMAC router · SrcMAC laptop · IP 192.168.1.5 → 93.184.216.34

THE RULE MACs change every hop; IPs persist end-to-end (until NAT rewrites them in Phase 4!).

4 Phase 4 — TCP Handshake (+ NAT at the edge)

The handshake (to 93.184.216.34:443)

SYN (seq=x) → 1 RTT total
← SYN-ACK (seq=y, ack=x+1)
ACK (ack=y+1) → ESTABLISHED

NAT happens right here

laptop: 192.168.1.5:52341 → 93.184.216.34:443
router: 203.0.113.5:10001 → 93.184.216.34:443
table: 10001 ↔ 192.168.1.5:52341

5 Phase 5 — TLS 1.3 Handshake (1-RTT)

The messages

ClientHello → TLS 1.3 · cipher suites · SNI: www.example.com · random
← ServerHello + Certificate (+chain) + Finished
Client Finished → ENCRYPTED CHANNEL ✓

- **SNI** — which domain's cert to serve (many sites per IP)
- **Certificate** — public key + domain, signed up a chain to a trusted root CA
- **Forward secrecy** — fresh ephemeral DH keys per session; leak later ≠ decrypt past
- **0-RTT** — returning clients send data immediately (replay-risk caveat)

Diffie-Hellman — the paint analogy

public: YELLOW paint
Alice secret RED · Bob secret BLUE
1 Alice: Y+RED=ORANGE → sends
2 Bob: Y+BLUE=GREEN → sends
3 Alice: GREEN+RED=BROWN
Bob: ORANGE+BLUE=BROWN
Same colour — secret never travelled!
Eve saw Y, O, G — can't unmix.

$$A = g^a \bmod p \quad B = g^b \bmod p \quad \cdot$$
$$\text{shared} = g^{(ab)} \bmod p$$

attacker can't compute $g^{(ab)}$ from g^a , g^b — discrete logarithm problem

WHY TLS NEEDS TCP TLS gives **confidentiality & integrity**, not delivery. It still leans on TCP for reliable, ordered record transport — that's why it runs *after* the handshake, *inside* the connection.

6 Phase 6 — The HTTP Request (encrypted)

THE REQUEST

```
GET /products?id=42 HTTP/1.1
Host: www.example.com
User-Agent: Mozilla/5.0 ...
Accept-Encoding: gzip, br
Cookie: session_id=abc123
Connection: keep-alive
```

INSIDE TLS

PACKET STACK LEAVING THE LAPTOP

L7 → L2

HTTP GET	L7 app
TLS record (AES)	L6 presentation
TCP :52341 → :443	L4 transport
IP .5 → .34	L3 network
ETH MAC→MAC	L2 link

7 Phase 7 — Server Side: LB → App → Response

1 Load balancer

Terminates TLS (**SSL offloading** — backends see plain HTTP), picks a backend, adds X-Forwarded-For: 203.0.113.5 so the app logs the real client.

2 Web app

Route /products → auth via session_id → DB query → render HTML.

3 Response

200 OK · Content-Encoding: gzip · Cache-Control: max-age=300 · ETag · Set-Cookie (Secure; HttpOnly) — compressed body.

8 Phase 8 — The Ride Home & Phase 9 — Close

```
LB re-encrypts → TCP src 93.184.216.34:443 → dst 203.0.113.5:10001
Router NAT reverse: 10001 → 192.168.1.5:52341 → LAN delivery
Laptop: TCP reassembles → TLS decrypts → HTTP parses → gunzip → RENDER 🚀
Sub-resources reuse the same TCP connection (keep-alive / HTTP-2 multiplexing)
```

```
Close: FIN → ACK → FIN → ACK → client TIME_WAIT ~60 s → NAT entry expires
```

The full timeline (typical, ~100 ms RTT to server)

- **0–5 ms Browser + OS cache misses**
conditional checks fail — we must go to the network.
- **5–70 ms DNS resolution**
resolver → root → TLD → authoritative → 93.184.216.34.
- **70–75 ms ARP + SYN out**
gateway MAC cached; SYN leaves, NAT entry created.
- **75–177 ms TCP handshake**
SYN-ACK back, ACK out — ESTABLISHED (1 RTT).
- **180–283 ms TLS 1.3**
ClientHello → ServerHello+cert → Finished (1 RTT).

- **285–400 ms Request + server work**
GET travels encrypted; LB, app, DB do their thing.
- **400–500 ms Response arrives**
decrypt, decompress, render HTML; 600–800 ms → CSS/JS/images done (often via CDN).

OPTIMISATIONS Second visit ≈ 50 ms because **cache + keep-alive + session resumption** skip DNS, handshake and most transfer. That's the exam's "why is the second load faster" answer, phase by phase.

9 Everything Maps — Protocol × Layer × Lecture

Phase	Protocol	OSI layer	Lecture	Phase	Protocol	Layer	Lec
Get an IP	DHCP	L7/L3	L11	Encryption	TLS	L6	L13
Gateway MAC	ARP	L2	L11	Web request	HTTP/1.1	L7	L9
Name → IP	DNS	L7	L9	Between ASes	BGP	L3	L8
Reliable pipe	TCP	L4	L10	Inside ISP	OSPF	L3	L8
Address sharing	NAT/PAT	L3	L4·L11	Packet delivery	IP	L3	L3·L7
Frame delivery	Ethernet	L2	L2	Local delivery	ARP	L2	L11

When it breaks — the troubleshooting map

Symptom	Phase	Tool	Typical fix
ERR_NAME_NOT_RESOLVED	DNS	dig	fix resolver / flush cache
ERR_CONNECTION_REFUSED	TCP	ss -tulpn	service down / wrong port
SSL_ERROR	TLS	openssl s_client	expired cert / wrong hostname
404 / 403	HTTP/app	devtools	path / permissions
500	App	server logs	bug in code
Slow TTFB	Server	monitoring	DB queries, caching
Intermittent drops	Path	mtr	loss mid-path → ISP

LECTURE 13 — RAPID REVISION CARD (THE WHOLE COURSE ON ONE CARD)

9 phases: cache → DNS → ARP → TCP → TLS → HTTP → server → return → close.

Cache chain: memory → disk (+ETag/304) → OS → resolver.

DNS: resolver walks root→TLD→auth; TTL 300 caches it.

ARP: gateway MAC (cache or broadcast). MACs change per hop, IPs persist till NAT.

TCP = 1 RTT handshake; **TLS 1.3** = 1 more RTT; both inside ~200 ms for 100 ms RTT.

DH: public g^a , g^b → shared $g^{(ab)}$; secrets never transmitted; KDF → AES keys.

LB: SSL offload + X-Forwarded-For; **NAT:** port table both directions.

Second visit fast: cache hits + keep-alive + TLS resumption skip phases 1,2,4,5.

TTFB = request-out → first-byte-in (RTT + server time).

Close: FIN/ACK×2, TIME_WAIT, NAT entry expires.



Important Questions — Exam Practice

A. Multiple Choice

Q1. A 304 Not Modified response means the browser sent:

MCQ

- a) Cache-Control: no-store
- b) If-None-Match with an ETag
- c) a fresh POST
- d) nothing — server guessed

Q2. TLS runs after the TCP handshake because:

MCQ

- a) DNS must finish first
- b) TLS records need TCP's reliable, ordered transport
- c) NAT requires it
- d) certificates use UDP

Q3. Why does the load balancer add X-Forwarded-For?

MCQ

- a) speed
- b) TLS encryption
- c) preserve the real client IP after proxying
- d) DNS

Q4. On the second visit to a page, which phases are typically skipped?

MCQ

- a) render only
- b) DNS + full TLS (cache, keep-alive, resumption)
- c) HTTP request
- d) TCP close

Q5. If the NAT entry for an active TCP flow expires mid-stream:

MCQ

- a) nothing happens
- b) return packets have no table entry → dropped; connection dies
- c) TCP re-handshakes automatically
- d) TLS renegotiates

B. Short Answer (2–4 marks)

Q6. List the nine phases of an HTTPS request in order with one line each.

SHORT

Q7. Why can't the browser skip the TCP handshake when TLS will encrypt anyway?

SHORT

Q8. What is SNI and what problem does it solve?

SHORT

Q9. Explain forward secrecy in one sentence using the paint analogy.

SHORT

Q10. What is TTFB and which two components make it up?

SHORT

Q11. Why don't CSS/JS requests after the HTML need new handshakes?

SHORT

C. Long Answer (5–8 marks)

Q12. The big one: trace <https://www.example.com> end-to-end with a diagram — cache, DNS, ARP, TCP, TLS, HTTP, NAT, LB, response, close — naming the protocol, OSI layer and lecture for each phase.

LONG

Q13. Explain the TLS 1.3 handshake AND the Diffie-Hellman exchange (paint + math). Why can't an eavesdropper compute the session key?

LONG

Q14. Reconstruct the ~500 ms timeline from this lecture and explain how caching, keep-alive and session resumption collapse it to ~50 ms.

LONG

Q15. "Everything in this course happens in one click." Prove it: map all 12 lectures to phases of the journey (table form), and identify the three most common failure points with tools.

LONG

ANSWER KEY

A1. b — conditional GET with the cached ETag.

A2. b — TLS is a user of TCP's byte stream.

A3. c — otherwise logs/apps only ever see the LB's IP.

A4. b — cached resources, reused connection, resumed TLS.

A5. b — replies arrive at the public IP:port with no mapping → dropped.

A6. Cache → DNS → ARP → TCP → TLS → HTTP request → server/LB → response → close.

A7. TLS rides inside TCP; without the byte stream there is nothing to encrypt over.

A8. ClientHello carries the hostname so the server presents the right certificate — many HTTPS sites per IP.

A9. Fresh ephemeral keys per session mean a future private-key leak can't un-mix past sessions' colours.

A10. Time-to-first-Byte = network RTT + server processing (LB+app+DB).

A11. keep-alive reuses the established connection; HTTP/2 multiplexes many streams over it.

A12–A15. The lecture IS the answer key — reproduce the phase diagram + mapping table + timeline with the optimisation column.